

Exercise 3: Specification-Guided Test Improvement

Lavanika Srinivasaraghavan
CS 520 – Fall 2025

GitHub Repository

Repository: <https://github.com/lava-nika/CS520-Exercise3-Specification-Guided-Test-Improvement>

See above GitHub repository's README.md for instructions to reproduce Coverage Results and steps to generate HTML Coverage Report.

Part 1: Generate, Evaluate, and Refine Specifications

In this part I have evaluated LLM's (ChatGPT 5.1) ability to generate formal specifications for two functions from the HumanEval dataset, which are given below.

Problem 4: is_prime (HumanEval/31)

Function Description

```
def is_prime(n: int) -> bool
```

Purpose: Return true if a given number is prime, and false otherwise.

A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself.

LLM Prompt used

None

Problem description: Write a function that checks whether a given number is prime.

A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself.

Method signature: `def is_prime(n: int) -> bool`

Please write formal specifications as Python assertions that describe the correct behavior of this method.

Let 'res' denote the expected return value of 'is_prime(n)'.

Do not call 'is_prime()' in your assertions.

Do not use methods with side effects such as `System.out.println`, file I/O, random number generation, or timing functions.

Express the relationship between 'n' and 'res' using pure arithmetic and boolean logic only.

Generate about 5 distinct, logically correct specifications that capture:

1. Numbers less than 2 are not prime ($n < 2$ implies not prime)
2. Number 2 is prime (special case)
3. Even numbers greater than 2 are not prime
4. If result is True, n should not be divisible by any number from 2 to n-1
5. If result is False and $n \geq 2$, there exists a divisor between 2 and n-1

Format each specification as:

```
# Specification N: <description>
assert <condition>, "Explanation"
```

Important: Use valid Python syntax only. The variable 'n' is an integer, 'res' is a boolean.

Generated assertions (Before correction)

Python

Specification 1: Numbers less than 2 are not prime

```
assert not res if n < 2 else True, "Numbers < 2 must not be classified as prime"
```

Specification 2: Number 2 is prime

```
assert (res == True) if n == 2 else True, "2 must be classified as prime"
```

Specification 3: Even numbers > 2 are not prime

```
assert (not res) if (n > 2 and n % 2 == 0) else True, "Even numbers greater than 2 are not prime"
```

Specification 4: If res is True, n has no divisors in [2, n-1]

```
assert (all(n % d != 0 for d in range(2, n))) if res else True, \
    "If res is True, n must not be divisible by any number between 2 and n-1"
```

Specification 5: If res is False and n >= 2, then n has some divisor in [2, n-1]

```
assert (any(n % d == 0 for d in range(2, n))) if (not res and n >= 2) else True, \
    "If res is False and n >= 2, there must exist a divisor between 2 and n-1"
```

Evaluation table: is_prime

#	Assertion	Correct	Issue	Corrected Version
1	assert not res if n < 2 else True	NO	Invalid conditional expression - should use	assert (n >= 2) or (not res)

			boolean implication	
2	<code>assert (res == True) if n == 2 else True</code>	NO	Invalid conditional expression - should use boolean implication	<code>assert (n != 2) or res</code>
3	<code>assert (not res) if (n > 2 and n % 2 == 0) else True</code>	NO	Invalid conditional expression - should use boolean implication	<code>assert (n <= 2) or (n % 2 != 0) or (not res)</code>
4	<code>assert (all(...)) if res else True</code>	NO	Invalid conditional expression - should use boolean implication	<code>assert (not res) or all(n % d != 0 for d in range(2, n))</code>
5	<code>assert (any(...)) if (not res and n >= 2) else True</code>	NO	Invalid conditional expression - should use boolean implication	<code>assert res or (n < 2) or any(n % d == 0 for d in range(2, n))</code>

Accuracy Rate: 0 / 5 = 0%

Corrected assertions: `is_prime`

Python

Specification 1: Numbers less than 2 are not prime

```
assert (n >= 2) or (not res), "Numbers < 2 must not be classified as prime"
```

Specification 2: Number 2 is prime

```
assert (n != 2) or res, "2 must be classified as prime"
```

Specification 3: Even numbers > 2 are not prime

```
assert (n <= 2) or (n % 2 != 0) or (not res), "Even numbers greater than 2 are not prime"
```

Specification 4: If res is True, n has no divisors in [2, n-1]

```

assert (not res) or all(n % d != 0 for d in range(2, n)), \
    "If res is True, n must not be divisible by any number between 2 and n-1"

# Specification 5: If res is False and n >= 2, then n has some divisor in [2, n-1]

assert res or (n < 2) or any(n % d == 0 for d in range(2, n)), \
    "If res is False and n >= 2, there must exist a divisor between 2 and n-1"

```

Problem 10: make_palindrome (HumanEval/10)

Function Description

```
def make_palindrome(string: str) -> str
```

Purpose: Find the shortest palindrome that begins with a supplied string.

Algorithm: Find the longest postfix of supplied string that is a palindrome, then append to the end of the string the reverse of the string prefix that comes before the palindromic suffix.

LLM Prompt used

None

Problem description: Find the shortest palindrome that begins with a supplied string.

Algorithm: Find the longest postfix of supplied string that is a palindrome, then append to the end of the string the reverse of the string prefix that comes before the palindromic suffix.

Method signature: def make_palindrome(string: str) -> str

Please write formal specifications as Python assertions that describe the correct behavior of this method.

Let 'res' denote the expected return value of 'make_palindrome(string)'.

Do not call 'make_palindrome()' in your assertions.

Do not use methods with side effects such as System.out.println, file I/O, random number generation, or timing functions.

Express the relationship between 'string' and 'res' using pure arithmetic and boolean logic only.

Generate about 5 distinct, logically correct specifications that capture:

1. Empty string handling (when string is empty, res should be empty)
2. The result must be a palindrome (res == res[::-1])
3. The result must start with the original string (res starts with string)
4. Length constraints (res length relative to input length)
5. Minimality property (res is the shortest palindrome starting with string)

Format each specification as:

Specification N: <description>

assert <condition>, "Explanation"

Important: Use valid Python syntax only. Use only pure boolean and string operations.

Generated assertions (Before correction)

Python

Specification 1: Empty string handling

```
assert (res == "") if string == "" else True, \
    "If the input string is empty, the result must also be an empty string"
```

Specification 2: The result must be a palindrome

```
assert res == res[::-1], \
    "The result must be a palindrome (equal to its own reverse)"
```

Specification 3: The result must start with the original string

```
assert res.startswith(string), \
    "The result must begin with the original input string"
```

Specification 4: Length constraints relative to input

```
assert (string == "" and len(res) == 0) or \
    (string != "" and len(string) <= len(res) <= 2 * len(string)), \
    "Result length must be at least the input length and at most twice the input length"
```

Specification 5: Minimality

```
assert all(
    not (prefix.startswith(string) and prefix == prefix[::-1])
    for prefix in (res[:k] for k in range(len(string), len(res)))
), \
    "There must be no shorter prefix of res that is itself a palindrome starting with the input string"
```

Evaluation table: make_palindrome

#	Assertion	Correct	Issue	Corrected Version
1	<code>assert (res == "") if string == "" else True</code>	NO	Invalid conditional expression - should use boolean implication	<code>assert (string != "") or (res == "")</code>
2	<code>assert res == res[::-1]</code>	YES	None	N/A
3	<code>assert res.startswith(string)</code>	YES	None	N/A
4	<code>assert (string == "" and len(res) == 0) or (string != "" and len(string) <= len(res) <= 2 * len(string))</code>	YES	None	N/A
5	<code>assert all(not (prefix.startswith(string) and prefix == prefix[::-1]) for prefix in (res[:k] for k in range(len(string), len(res))))</code>	YES	None	N/A

Accuracy Rate: 4 / 5 = **80%**

Corrected assertions: make_palindrome

Python

Specification 1: Empty string handling (CORRECTED)

```
assert (string != "") or (res == ""), \
```

"If the input string is empty, the result must also be an empty string"

```

# Specification 2: The result must be a palindrome

assert res == res[::-1], \

    "The result must be a palindrome (equal to its own reverse)"


# Specification 3: The result must start with the original string

assert res.startswith(string), \

    "The result must begin with the original input string"


# Specification 4: Length constraints relative to input

assert (string == "" and len(res) == 0) or \

    (string != "" and len(string) <= len(res) <= 2 * len(string)), \

    "Result length must be at least the input length and at most twice the\ninput length"


# Specification 5: Minimality

assert all(

    not (prefix.startswith(string) and prefix == prefix[::-1])

    for prefix in (res[k:] for k in range(len(string), len(res)))

), \

    "There must be no shorter prefix of res that is itself a palindrome\nstarting with the input string"

```

Accuracy summary for Part 1

Problem	Correct	Total	Accuracy
is_prime	0	5	0%
make_palindrome	4	5	80%
Overall	4	10	40%

The main issue identified was:

The LLM consistently used this pattern:

```
Python
assert <value> if <condition> else True
```

This is a conditional expression, not proper boolean logic for specifications. Even though it is syntactically correct, it does not correctly express logical implications.

Correct pattern: Logical implication "if A then B" should be expressed as:

```
Python
assert (not A) or B
```

This means "either A is false, or B is true" which is logically equivalent to "A implies B."

Part 2: Use Specifications to Guide Test Improvement

In this part I have used the corrected specifications from Part 1 to generate test cases using ChatGPT 5.1, and then compared the coverage with baseline HumanEval tests from Exercise 2.

LLM Prompts for test generation

Prompt 1: is_prime test generation

```
None
I have the following corrected formal specifications for the is_prime
function:

Function signature: def is_prime(n: int) -> bool

Corrected Specifications (as assertions where 'res' is the return value):
```

```

# Specification 1: Numbers less than 2 are not prime
assert (n >= 2) or (not res), "Numbers < 2 must not be classified as prime"

# Specification 2: Number 2 is prime
assert (n != 2) or res, "2 must be classified as prime"

# Specification 3: Even numbers > 2 are not prime
assert (n <= 2) or (n % 2 != 0) or (not res), "Even numbers greater than 2
are not prime"

# Specification 4: If res is True, n has no divisors in [2, n-1]
assert (not res) or all(n % d != 0 for d in range(2, n)), \
    "If res is True, n must not be divisible by any number between 2 and
n-1"

# Specification 5: If res is False and n >= 2, then n has some divisor in
[2, n-1]
assert res or (n < 2) or any(n % d == 0 for d in range(2, n)), \
    "If res is False and n >= 2, there must exist a divisor between 2 and
n-1"

```

Please generate pytest test cases that validate these specifications. Each test should:

1. Call `is_prime` with specific input values
2. Check that the result satisfies the relevant specifications
3. Be clearly labeled as spec-guided tests
4. Cover edge cases and boundary conditions implied by the specifications

Generate about 5-7 test functions in pytest format. Include:

- Test for $n < 2$ (Spec 1)
- Test for $n = 2$ (Spec 2)
- Test for even numbers > 2 (Spec 3)
- Test for prime numbers (Spec 4)
- Test for composite numbers (Spec 5)
- Edge cases

Format:

```
def test_spec_<number>_<description>():  
    # Test for Specification <number>  
    <test code>
```

Prompt 2: make_palindrome test generation

None

I have the following corrected formal specifications for the make_palindrome function:

Function signature: `def make_palindrome(string: str) -> str`

Corrected Specifications (as assertions where 'res' is the return value):

Specification 1: Empty string handling

```
assert (string != "") or (res == ""), \
```

"If the input string is empty, the result must also be an empty string"

```

# Specification 2: The result must be a palindrome
assert res == res[::-1], \
    "The result must be a palindrome (equal to its own reverse)"

# Specification 3: The result must start with the original string
assert res.startswith(string), \
    "The result must begin with the original input string"

# Specification 4: Length constraints relative to input
assert (string == "" and len(res) == 0) or \
    (string != "" and len(string) <= len(res) <= 2 * len(string)), \
    "Result length must be at least the input length and at most twice the input length"

# Specification 5: Minimality – no shorter prefix of res is a valid
# palindrome starting with string
assert all(
    not (prefix.startswith(string) and prefix == prefix[::-1])
    for prefix in (res[:k] for k in range(len(string), len(res)))
), \
    "There must be no shorter prefix of res that is itself a palindrome
    starting with the input string"

```

Please generate pytest test cases that validate these specifications. Each test should:

1. Call `make_palindrome` with specific input values
2. Check that the result satisfies the relevant specifications
3. Be clearly labeled as spec-guided tests

4. Cover edge cases and boundary conditions implied by the specifications

Generate about 5-7 test functions in pytest format. Include:

- Test for empty string (Spec 1)
- Test that result is palindrome (Spec 2)
- Test that result starts with input (Spec 3)
- Test length constraints (Spec 4)
- Test minimality (Spec 5)
- Edge cases

Format:

```
def test_spec_<number>_<description>():  
    # Test for Specification <number>  
    <test code>
```

Generated Spec-Guided tests

is_prime (6 tests)

Python

```
def test_spec_1_numbers_less_than_two():  
    """Test Specification 1: Numbers less than 2 are not prime"""  
    test_values = [-10, -1, 0, 1]  
    for n in test_values:  
        res = is_prime(n)  
        assert (n >= 2) or (not res)
```

```

def test_spec_2_two_is_prime():
    """Test Specification 2: Number 2 is prime"""
    n = 2
    res = is_prime(n)
    assert (n != 2) or res
    assert res is True

def test_spec_3_even_numbers_greater_than_two():
    """Test Specification 3: Even numbers > 2 are not prime"""
    test_values = [4, 6, 10, 100, 102]
    for n in test_values:
        res = is_prime(n)
        assert (n <= 2) or (n % 2 != 0) or (not res)
        assert res is False

def test_spec_4_prime_numbers_have_no_divisors():
    """Test Specification 4: If res is True, n has no divisors in [2,
n-1]"""
    prime_candidates = [2, 3, 5, 7, 11, 29]
    for n in prime_candidates:
        res = is_prime(n)
        assert (not res) or all(n % d != 0 for d in range(2, n))
        assert res is True

def test_spec_5_composite_numbers_have_divisors():
    """Test Specification 5: If res is False and n >= 2, then n has some
divisor"""

```

```

composite_values = [4, 6, 9, 15, 21, 100]

for n in composite_values:
    res = is_prime(n)

    assert res or (n < 2) or any(n % d == 0 for d in range(2, n))

    assert res is False

def test_spec_edge_negative_and_large_values():
    """Spec-guided edge cases"""
    test_values = [-5, -2, 0, 1, 97, 99]

    for n in test_values:
        res = is_prime(n)

        assert (n >= 2) or (not res)

        if n >= 2:
            assert (not res) or all(n % d != 0 for d in range(2, n))

            assert res or any(n % d == 0 for d in range(2, n))

```

make_palindrome (6 tests)

Python

```

def test_spec_1_empty_string():
    """Test Specification 1: Empty string handling"""
    string = ''

    res = make_palindrome(string)

    assert (string != "") or (res == "")

    assert res == ""

def test_spec_2_result_is_palindrome_for_various_inputs():

```

```

    """Test Specification 2: The result must be a palindrome"""

    test_cases = ['', 'a', 'ab', 'abc', 'racecar']

    for string in test_cases:

        res = make_palindrome(string)

        assert res == res[::-1]


def test_spec_3_result_starts_with_input():

    """Test Specification 3: The result must start with the original
    string"""

    test_cases = ['', 'x', 'cat', 'hello']

    for string in test_cases:

        res = make_palindrome(string)

        assert res.startswith(string)


def test_spec_4_length_constraints():

    """Test Specification 4: Length constraints relative to input"""

    test_cases = ['', 'a', 'ab', 'xyz']

    for string in test_cases:

        res = make_palindrome(string)

        assert (string == "" and len(res) == 0) or \
            (string != "" and len(string) <= len(res) <= 2 * len(string))


def test_spec_5_minimality_property():

    """Test Specification 5: Minimality"""

    test_cases = ['', 'x', 'ab', 'cat', 'racecar']

    for string in test_cases:

        res = make_palindrome(string)

```

```

    assert all(
        not (prefix.startswith(string) and prefix == prefix[::-1])
        for prefix in (res[:k] for k in range(len(string), len(res)))
    )

def test_spec_edge_cases_various_strings():
    """Spec-guided edge cases for various strings"""
    test_cases = ['', 'a', 'ab', 'abc', 'abcd', 'palindrome']
    for string in test_cases:
        res = make_palindrome(string)
        assert (string != "") or (res == "")
        assert res == res[::-1]
        assert res.startswith(string)

```

Coverage Comparison Results

Before/after coverage table

Problem	Old Stmt %	New Stmt %	Old Branch %	New Branch %
is_prime	100%	100%	100%	100%
make_palindrome	100%	100%	100%	100%

Detailed coverage analysis

Problem 4: is_prime

Baseline Tests (HumanEval)

- Test function: 1 test with 13 assertions
- Test cases: `is_prime(1)`, `is_prime(4)`, `is_prime(5)`, `is_prime(6)`, `is_prime(11)`, `is_prime(17)`, `is_prime(61)`, `is_prime(101)`, `is_prime(13441)`, `is_prime(5*17)`, `is_prime(11*7)`, `is_prime(13441*19)`
- Statement Coverage: **100%** (24 statements)
- Branch Coverage: **100%** (6/6 branches)

Spec-Guided Tests

- Test functions: 6 focused test functions
- Test cases: Multiple values across edge cases (negative, 0, 1, 2, even numbers, primes, composites)
- Statement Coverage: **100%**
- Branch Coverage: **100%**

Coverage Improvement: 0% (both achieve full coverage)

Problem 10: make_palindrome

Baseline Tests (HumanEval)

- Test function: 1 test with 5 assertions
- Test cases: ' ', 'x', 'xyz', 'xyx', 'jerry'
- Statement Coverage: **100%** (18 statements)
- Branch Coverage: **100%** (4/4 branches)

Spec-Guided Tests

- Test functions: 6 focused test functions
- Test cases: Empty string, single char, various multi-char strings, already palindromic
- Statement Coverage: **100%**
- Branch Coverage: **100%**

Coverage Improvement: 0% (both achieve full coverage)

Case-specific insight per problem (Why coverage did not improve):

Problem 4: is_prime:

The baseline HumanEval tests already achieved 100% coverage because of

1. **Comprehensive test suite.** The 13 test cases covered all possible code paths which were
 - `n < 2` check (tested with `n=1`)
 - Loop iteration for divisor checking (various primes and composites)
 - Both True and False return paths
2. **Simple control flow:** The `is_prime` function has only 6 branches
 - `if n < 2: return False`
 - `for k in range(2, n):`
 - `if n % k == 0: return False`
 - `return True`
3. **Overlapping coverage:** The specifications formalized properties which were already tested like
 - Spec 1 (`n < 2`) -> Tested by `is_prime(1) == False`
 - Spec 2 (2 is prime) -> Implicitly tested by prime test cases
 - Spec 3 (even numbers) -> Tested by `is_prime(4)`, `is_prime(6)`

- Specs 4 & 5 (divisor checking) -> Tested by all prime/composite cases

Problem 10: make_palindrome

The baseline tests achieved 100% coverage because:

1. **Sufficient examples were present.** 5 test cases covered all paths
 - Empty string handling
 - Single character (already palindrome)
 - String requiring suffix addition ('xyz' -> 'xyzyx')
 - Already palindromic string ('xyx')
 - Longer string ('jerry' -> 'jerryrrej')
2. **Simple logic:** The function has a straightforward control flow.
 - Empty string check
 - While loop to find palindromic suffix
 - String concatenation
3. **Property Coverage:** All specifications were implicitly satisfied.
 - Spec 1 (empty) -> `make_palindrome('') == ''`
 - Spec 2 (palindrome) -> Result always palindrome by construction
 - Spec 3 (starts with) -> Verified by all test assertions
 - Specs 4 & 5 (length, minimality) -> Satisfied by algorithm design

From these we can see that the HumanEval tests were already thorough. Also the functions are simple and with limited branching so 100% coverage is achieved with basic tests itself. Also the formal specs captured properties which were already tested.